# Modelli e Linguaggi

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

18 febbraio 2025

# Modelli

## Progettazione basata su modelli

- La fase di progetto di un artefatto complesso si basa sull'uso di *modelli*
- Il modello consente di focalizzarsi sugli aspetti primari del problema e permette verifiche preliminari sul funzionamento

## Come modellare?

- Modelli fisici: riproduzioni in scala (e.g., dighe in miniatura)
- Modelli teorici/formali: oggetti matematici che consentono una visione astratta del fenomeno modellato

# Modelli formali

## Uso dei modelli formali

1. Formalizzazione del problema: ottenere una descrizione astratta dell'entità reale
2. Risoluzione del problema formalizzato
3. Interpretazione dei risultati alla luce del modello, nel contesto dell'entità reale $\rightarrow$ (ri-)valutazione delle scelte di progetto

## Quando un modello "funziona"?

- Un modello è *adeguato* se riflette correttamente il fenomeno modellato per gli aspetti che interessano
- Posso approssimare un cavallo a una sfera/punto materiale?
  - Per progettare una sella? Hmm... no.
  - Per calcolare quanto è robusto il pavimento della stalla? Non è così terribile

# Modelli dell'informatica

## Una "realtà" molto vicina al modello ...

- L'informatica è una delle discipline con maggior vicinanza tra il modello e la realtà di cui si occupa (il calcolo)
- Nella sua evoluzione i primi calcolatori sono stati la materializzazione diretta del modello di calcolo ("inversione" di tendenza: realtà ad immagine del modello)

## ... e i relativi vantaggi

- I modelli formali sono di grande aiuto nella progettazione di HW e SW
  - Garanzie di correttezza, efficienza, robustezza, sicurezza
- Certificazione del livello di formalizzazione dell'artefatto HW/SW inclusa in standard (ISO/IEC 15408, livelli EAL)

# Applicazioni dell'informatica

## L'informatica a sostegno di altre discipline

- L'informatica trova applicazioni in svariati contesti $\rightarrow$ un particolare calcolo effettuato è il modello di qualcosa di reale
- Il successo è determinato dalla flessibilità del calcolo astratto come modello di una sequenza di ragionamenti logici
- Spesso la difficoltà sta proprio nel *formulare* il modello (= definire il problema)!
- Cosa significa dare un modello (quantitativo) a:
  - estrazione di "informazione" da grandi moli di dati
  - garantire la sicurezza di un sistema rispetto a incidenti/dolo
  - generare testo/immagini rassomiglianti ad esempi "naturali"
  - rispondere ad una domanda facendo una sintesi di contenuti

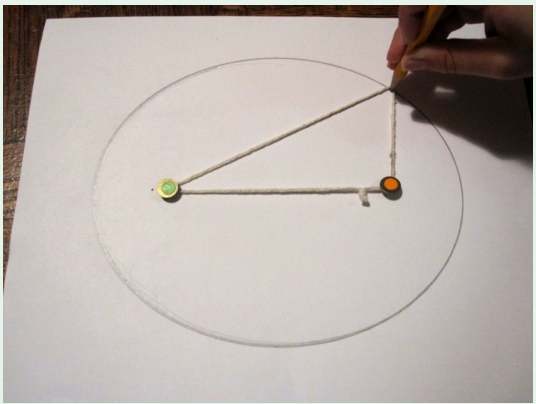
# Alcune provocazioni

## E. W. Dijkstra - premio Turing nel 1972

- *[...] in our area, perhaps more than everywhere else, mathematical elegance is not a dispensable luxury but decides between success and failure.*
- *It is nice to know the dictionary definition for the adjective "elegant" in the meaning "simple and surprisingly effective".*
- *Programming is one of the most difficult branches of applied mathematics; the poorer mathematicians had better remain pure mathematicians.*
- *If you want more effective programmers, you will discover that they should not waste their time debugging, they should not introduce the bugs to start with.*

# Una classificazione dei modelli - Ellisse

## Ellisse - Modello operativo



## Ellisse - Modello descrittivo

$$a, b, c, x, y \in \mathbb{R}$$

$$ax^2 + by^2 = c$$

# Una classificazione dei modelli - Ordinamento

## Modello operativo

Dato un insieme di $n$ elementi disordinati,

1. ripeti $n-1$ volte
   1. Trova l'elemento più piccolo tra quelli da ordinare
   2. Aggiungilo in coda agli elementi ordinati
2. aggiungi in coda l'ultimo elemento

## Modello descrittivo

La permutazione della sequenza $a$ data t.c.

$$\forall i, a[i] \leq a[i+1]$$

# Il linguaggio

## Un (il?) primo modello

- Il linguaggio è da sempre uno strumento per esprimere modelli
- Esempi di linguaggio
  - Lingue naturali: italiano, inglese, francese, giapponese
  - Linguaggi sintetici: C, Java, XML, JSON
  - Linguaggi grafici: cartelli stradali, simboli di lavaggio capi
  - Linguaggi acustici: musica, fischi degli arbitri sportivi

# Gli elementi del linguaggio

## Alfabeto

- Un linguaggio è definito su un *alfabeto*
- L'alfabeto è un insieme *finito* di simboli
- Esempi:
  - $\{a, b, c, \ldots, z\}$
  - $\{0, 1\}$
  - Codice Morse, Codice Baudot, Codice ASCII
  - $\{$𝅗𝅥, ♩, ♪, ... $\}$
  - $\{$, , , ... $\}$

# Gli elementi del linguaggio

## Stringa su un alfabeto $A$

- Stringa: sequenza *ordinata* e *finita* di elementi dell'alfabeto
- Esempio: a, `stringa`, `vicesindaco`, `klatubaradanikto`
- Lunghezza di una stringa $\rightarrow$ numero di elmenti: $|a| = 1,\ |stringa| = 7$
- Stringa nulla $\varepsilon$, $|\varepsilon| = 0$
- $A^*$ insieme di tutte le stringhe (inclusa $\varepsilon$) su $A$
  - Esempio: $A = \{0, 1\}, A^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 100, 101, \ldots\}$

# Operazioni tra stringhe

## Concatenazione

- L'operatore di concatenazione è il . (può essere omesso)
  - Con $x$=Bana, $y$=na, $z$=ch, si ha $x.y$=Banana, $x.z$=Banach
- L'operazione di concatenazione è binaria, associativa, non commutativa

## Monoide libero

- Dato un alfabeto $A$, $(A^*, .)$ è il *monoide libero* costruito su $A$
  - L'elemento neutro del monoide è la stringa nulla $\varepsilon$

# Linguaggio

## Linguaggio su un alfabeto $A$

- Un linguaggio $L$ su un alfabeto $A$ è un sottoinsieme di $A^*$
  - ... ovvero un insieme di *sequenze* di simboli di $A$ (parole)
- Esempi:
  1. l'italiano è un linguaggio su $A = \{a, b, c, \ldots, z, _\}$
  2. i files PDF sono un linguaggio su $A = \{0, 1\}$
  3. i numeri pari in base 4 sono un linguaggio su $A = \{0, 1, 2, 3\}$
  4. il DNA è un linguaggio codificabile su $A = \{C, G, A, T\}$
- n.b. anche se $|A| < \infty$, non necessariamente $|L| < \infty$ (vedi esempio 3)
- A seconda dell'alfabeto scelto, un linguaggio può modellare moltissime cose (è, in un certo senso, *universale*)

# Operazioni tra linguaggi

## Operazioni insiemistiche

- I linguaggi sono insiemi, valgono $\cup$ (unione), $\cap$ (inters.), $\setminus$ (differenza)
- Il complemento $\neg L = \bar{L}$ è definito rispetto ad $A^*$, ovvero $\neg L = A^* \setminus L$

## Concatenazione tra linguaggi

- Dati $L_1$ su $A_1$, $L_2$ su $A_2$, $L_1.L_2$, definito su $A_1 \cup A_2$ è:

$$L_1.L_2 = \{x.y \mid x \in L_1, y \in L_2\}$$

- Es.: Dati $L_1 = \{0,1\}^*, L_2 = \{a,b\}^*$ abbiamo $L_1.L_2 = \{\varepsilon, 0, 1, 0a, 011b, 0aba, \ldots\}$. Nota bene: $a1 \notin L_1.L_2$

# Operazioni tra linguaggi

## Alcune proprietà

- $L^n, n \in \mathbb{N}$, la concatenazione di $L$ con sè stesso $n$ volte
  - Definizione compatta: $L^0 = \{\varepsilon\}$, $L^n = L^{n-1}.L$
- N.b.: il linguaggio vuoto $L_1 = \varnothing \neq \{\varepsilon\} = L_2$ il linguaggio fatto dalla sola stringa vuota.
  - effetti collaterali includono $\{\varepsilon\}.L = L \neq L.\varnothing = \varnothing$

## Operatore +

- L'operatore + indica stringhe fatte concatenando *uno* o più elementi dell'insieme.
  - $A = \{0, 1\}, A^+ = \{0, 1, 00, 01, 10, \ldots\}$
- Per estensione: $L^* = \bigcup_{n=0}^{\infty} L^n$, $L^+ = \bigcup_{n=1}^{\infty} L^n$

# Operazioni tra linguaggi

## Nella pratica

- È possibile definire un formato di file come linguaggio (e.g., una pagina HTML)
- L'intersezione tra linguaggi ci indica se due formati sono "compatibili"
  - Se $L_1$ sono i documenti HTML 4 e $L_2$ quelli HTML 5, $L_1 \cap L_2$ sono i documenti corretti secondo entrambi gli standard
- La concatenazione di linguaggi consente di descrivere più agevolmente formati complessi
  - Formato di un pacchetto di dati su rete $L_{\text{header}}.L_{\text{dati}}.L_{\text{trailer}}$
  - Archivio di dati `tar` o `zip`: $L_{\text{indice}}.L_{\text{file}}^n$

# Usi del linguaggio

## Il linguaggio come formalismo espressivo

- Un linguaggio può essere usato per esprimere un problema:
  - "Trovare la miglior mossa successiva in una partita a scacchi"
  - "Trovare tre numeri interi positivi tali per cui $x^3 + y^3 = z^3$"
- L'insieme delle soluzioni di un problema è un linguaggio:
  - Risolvere il problema $\rightarrow$ calcolare (riconoscere) un $x \in L$
  - Es: "Questo programma C è sintatticamente corretto?"
    - Data $x$ dire se $x \in L_{\texttt{linguaggio C}}$
  - "Questa immagine digitale contiene un volto?"
  - "Questo brano digitalizzato è in quattro quarti?"
  - "Questa funzione è la derivata di $x^3 + x$?"

# Da un linguaggio all'altro

## Traduzioni

- Tradurre = mettere in corrispondenza parole di due linguaggi
- Formalmente, una traduzione è una mappa $\tau(\cdot) : L_1 \to L_2$
  - $L_1 = \{a, b\}^*, L_2 = \{c, d\}^*$, se $\tau$ mappa $a \mapsto c$, $b \mapsto d$, allora $\tau(ba) = dc$
  - $L_1 = \{0, 1\}^*, L_2 = \{0, 1\}^*$, se $\tau$ scambia $1$ con $0$, allora $\tau(010) = 101$

## Esempi pratici

- L'insieme delle soluzioni di un problema è un linguaggio:
  - Compressione/cambio di formato di un file
  - Compilazione di un programma in codice oggetto
  - Correzione ortografica/sintattica meccanizzato

# Conclusione

## Linguaggi come metodo espressivo

- Il linguaggio costituisce un mezzo per descrivere la natura di un problema e quella delle sue soluzioni
- Useremo la formalizzazione del linguaggio come elemento fondamentale per formalizzare il concetto di calcolo
- Il passaggio alla pratica è "più diretto di quanto sembri", dopotutto il calcolatore maneggia stringhe su un alfabeto binario...

# Automi a stati finiti

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

19 febbraio 2025

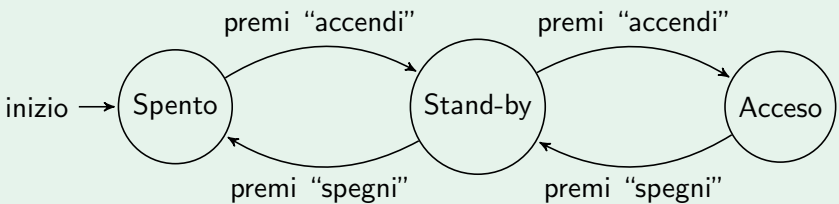
# Modelli operativi

## Un semplice modello di calcolo

- I modelli operativi di calcolo sono definiti come *macchine astratte*
- Modellano il calcolo come una serie di passi *discreti* da una condizione (*stato*) alla successiva
- Il primo (più semplice) che vediamo sono gli Automi a Stati Finiti (ASF, o Finite State Automata, FSA)
- Gli FSA hanno memoria del calcolo formata da un insieme finito di stati.
- Esempi: {marce di un'automobile},{fermo, passo, trotto, galoppo}, {in partenza, in viaggio, in arrivo}, $\{1, 2 \ldots, k\}$

# Modelli operativi

## Un modello a stati finiti di un televisore



# Formalizzazione

## Costituenti di un FSA

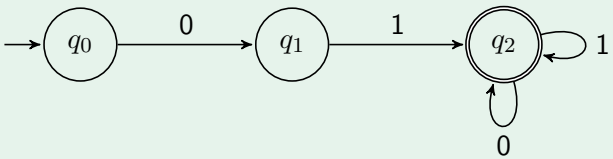
- $\mathbf{Q}$, l'insieme finito dei suoi stati
- $\mathbf{I}$, l'insieme finito (alfabeto) dei simboli in ingresso
- $\delta : \mathbf{Q} \times \mathbf{I} \rightarrow \mathbf{Q}$ la funzione di transizione: mappa una coppia (stato corrente, input) in uno stato di destinazione
- Serve definire un inizio della computazione: chiamiamo $q_0 \in \mathbf{Q}$ lo stato iniziale dell'automa ($q_0 =$ Spento, nell'esempio precedente)
  - Si indica con una freccia entrante nello stato iniziale non originata da un altro stato

# FSA Riconoscitore

## Riconoscere un linguaggio con un FSA

- Possiamo usare un FSA per riconoscere le parole di un linguaggio
- Definiamo l'insieme di stati finali $\mathbf{F} \subseteq \mathbf{Q}$
- Se l'automa, leggendo una stringa, partendo da $q_0$ termina in uno stato finale, la stringa appartiene al linguaggio

## Riconoscitore per $L = \{\text{stringhe che iniziano con } 01\}$



# Formalizzazione dell'accettazione

## Sequenza di mosse

- Formalizziamo una sequenza di mosse definendo $\delta^* : \mathbf{Q} \times \mathbf{I}^* \to \mathbf{Q}$, estensione di $\delta$ induttivamente:
  - **Base**: $\forall q \in \mathbf{Q},\ \delta^*(q, \varepsilon) = q$
  - **Passo**: $\delta^*(q, y.i) = \delta(\delta^*(q, y), i)$, con $i \in \mathbf{I}$ e $y \in \mathbf{I}^*$

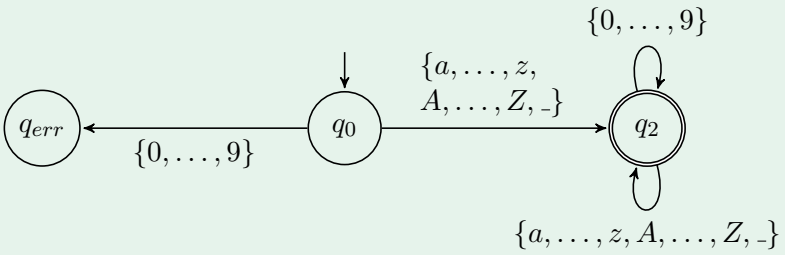
## Accettazione di un linguaggio

- Possiamo formalizzare l'accettazione di un linguaggio $L$ su $\mathbf{I}$, da parte di un FSA $(\mathbf{Q}, \mathbf{I}, \delta, q_0, \mathbf{F})$ come

$$x \in L \Leftrightarrow \delta^*(q_0, x) \in \mathbf{F}$$

# Riconoscitore degli identificatori del linguaggio C

**$L_{id}$ definito su $\mathbf{I} = \{a, b, \ldots, z, A, B, \ldots, Z, 0, \ldots, 9, _\}$**

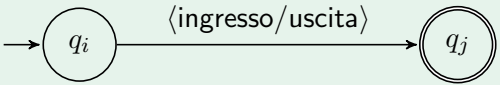



n.b. Arco etichettato con un insieme di $n$ elementi $\rightarrow$ abbreviazione per $n$ archi, uno per ogni simbolo (si possono omettere le graffe se non ambiguo)

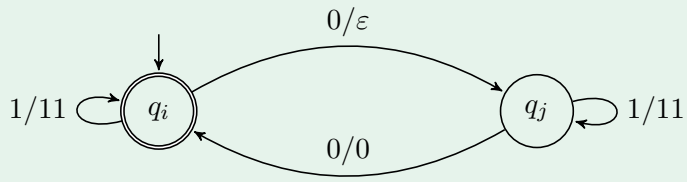
# Traduttori

## FSA traduttore

- Un FSA traduttore da $L_1$ a $L_2$ associa un simbolo letto e uno scritto a ogni transizione




## $L_1 \subset \{0,1\}^*$ stringhe con numero di "0" pari, $\tau$ : dimezza gli "0", raddoppia gli "1"



# Formalizzazione di un traduttore

## Elementi del traduttore

- Un FSA traduttore: 7-upla $\mathcal{A} : (\mathbf{Q}, \mathbf{I}, \delta, q_0, \mathbf{F}, \mathbf{O}, \eta)$
- $\langle \mathbf{Q}, \mathbf{I}, \delta, q_0, \mathbf{F} \rangle$ come nell'FSA riconoscitore
- $\mathbf{O}$: alfabeto di uscita
- $\eta : \mathbf{Q} \times \mathbf{I} \rightarrow \mathbf{O}^*$ funzione di traduzione

## Funzione di traduzione per stringhe $\eta^*$

- $\eta^* : \mathbf{Q} \times \mathbf{I}^* \rightarrow \mathbf{O}^*$ definita in maniera analoga
  - **Base** $\eta^*(q, \varepsilon) = \varepsilon$
  - **Passo** $\eta^*(q, y.i) = \eta^*(q, y).\eta(\delta^*(q, y), i)$, con $i \in \mathbf{I}, y \in \mathbf{I}^*$
- L'intero calcolo di traduzione è quindi formalizzato come

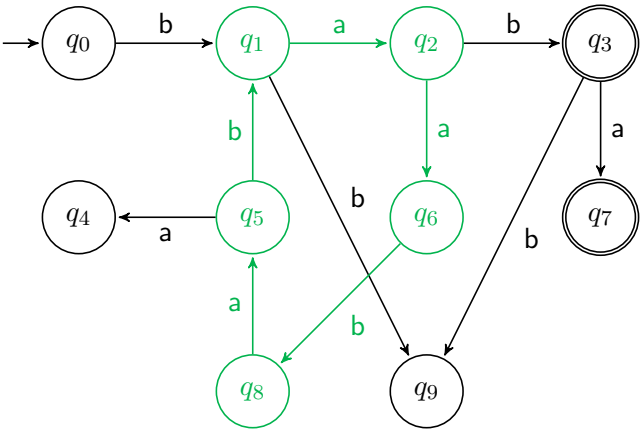
$$\tau(x) = \eta^*(q_0, x) \Leftrightarrow \delta^*(q_0, x) \in \mathbf{F}$$

# Analisi del modello FSA

## Alcune considerazioni

- Modello semplice, ma molto usato
  - E.g.: stati di esecuzione di un processo per il sistema operativo
- Questa semplicità ha un costo?
  - Lo quantificheremo più avanti (alcune calcoli non sono modellabili solo con un FSA)
- Una prima proprietà fondamentale: il *comportamento ciclico* degli FSA

# Esempio di ciclo




Esiste un ciclo $\delta^*(q_1, aabab) = q_1$ percorribile $0, 1, 2, \ldots, n$ volte

# Formalizzare il comportamento ciclico

## Pumping lemma

- Premessa: $\exists x \in L$, $L$ riconosciuto da un FSA, $|x| \geq |\mathbf{Q}|$
- Conseguenza: esistono $q \in \mathbf{Q}$, $w \in \mathbf{I}^+$ tali che $x = ywz$ con $y, z \in \mathbf{I}^*$ e $\delta^*(q, w) = q$
  - ovvero esiste una sottostringa di $x$ che viene riconosciuta dall'automa effettuando un'iterazione su un ciclo di stati
- Dal pumping lemma consegue che $yw^n z \in L, \forall n \geq 0$
  - Segue dal poter effettuare zero o più iterazioni del ciclo
- Da cui "pumping lemma", posso "gonfiare" il numero di $w$

# Conseguenze del Pumping lemma

## Proprietà dei linguaggi riconosciuti da FSA

- Posso dire se $L = \varnothing$
  - $\exists x \in L \Rightarrow \exists y \in L, |y| < |\mathbf{Q}|$: se una parola ha "cicli in riconoscimento" li elimino $\rightarrow$ posso dare in pasto le $y$ all'FSA (sono finite) e verificare se almeno una $\in L$
- Posso dire se $|L| = \infty$
  - $\exists x \in L, |\mathbf{Q}| \leq |x| < 2|\mathbf{Q}|$ implica che $x$ abbia un ciclo in riconoscimento
- N.B. aver un modo, *in generale* (con altri modelli di calcolo $\neq$ FSA), di dire se $x \in L$ non implica saper rispondere queste due domande

# Conseguenze pratiche del Pumping lemma

## Proprietà "utili" di un linguaggio

- Mi interessa aver definito un linguaggio (di programmazione, di descrizione dati) consistente di nessuna parola?
- Mi interessa aver definito un linguaggio di programmazione in cui poter scrivere solo un numero finito di programmi?

# Limitazioni degli FSA

## Riconoscere strutture a parentesi

- $L = \{a^n b^n, n \geq 0\}$ è riconosciuto da un FSA?
- Intuizione: no. Dimostriamolo per assurdo
- sia $x \in L, x = a^m b^m, \frac{m}{2} > |Q|$, applicando il pumping lemma abbiamo che $x = ywz$, con $w$ che avere una tra le seguenti forme
  - $w = a^k$, pompando $w$ ottengo $\forall r \in \mathbb{N}, a^{m-k} a^{r \cdot k} b^m \in L$, ↯
  - $w = b^k$, pompando $w$ ottengo $\forall r \in \mathbb{N}, a^m b^{r \cdot k} b^{m-k} \in L$, ↯
  - $w = a^k b^h$, pompando $w$ ottengo $\forall r \in \mathbb{N}, a^{m-k} (a^k b^h)^r b^{m-h} \in L$, ↯

# Limitazioni degli FSA

## Verso modelli più potenti

- Intuitivamente: per "contare" un $n$ arbitrariamente grande, occorre una quantità di memoria arbitrariamente grande
- Riconoscere strutture a parentesi (HTML,XML,linguaggi di programmazione) non è fattibile da un FSA
- Anche modellare un calcolatore fisico (che è un FSA) come tale può essere scomodo/intrattabile ($2^{2^{44}}$ stati)
- Sarà necessario "estendere" gli FSA per renderli più efficaci

# Il concetto di chiusura

## Chiusura (algebrica)

- Dati un insieme $\mathbf{S}$, ed un'operazione definita sui suoi elementi si dice che $\mathbf{S}$ è *chiuso* rispetto all'operazione, se il risultato di dell'applicarla ad un elemento di $\mathbf{S}$ è contenuto in $\mathbf{S}$

## Esempi

- I numeri naturali (insieme $\mathbb{N}$) sono chiusi rispetto alla somma, ma non rispetto alla sottrazione
- L'insieme dei convogli ferroviari è chiuso per concatenazione

# Chiusura nei linguaggi

## Chiusura di famiglie di linguaggi

- Famiglia di linguaggi: un insieme $\mathbb{L}$ i cui elementi sono linguaggi, $\mathbf{L} = \{L_i\}$
- $\mathbf{L}$ è chiusa rispetto a un'operazione (binaria) $\star$ se $\forall L_1, L_2 \in \mathbf{L}$ vale $L_1 \star L_2 \in \mathbf{L}$

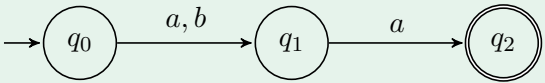
## Linguaggi regolari

- La famiglia di linguaggi riconoscibili con un FSA è la famiglia dei linguaggi *regolari*, $\mathbf{R}$ o $\mathrm{REG}$
- $\mathbf{R}$ è chiusa rispetto a $\cup, \cap, \neg, \setminus$, alla concatenazione, a $*$ e $+$

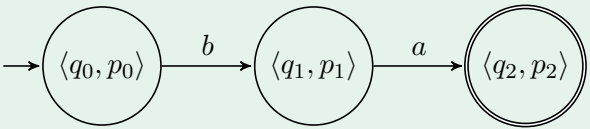
# Intersezione tra linguaggi

## Combinare gli FSA

Dati due FSA riconoscitori

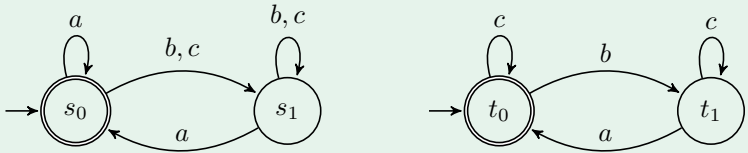



Ottengo il riconoscitore del linguaggio intersezione facendoli funzionare "insieme": è possibile una transizione solo se c'è in entrambi

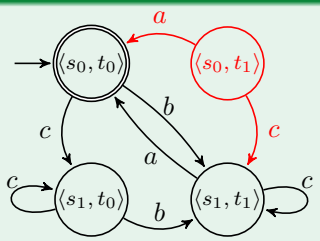


# Intersezione: esempio

## Automi "sorgente"




## Automa intersezione




- Lo stato in rosso non è raggiungibile dallo stato iniziale $\rightarrow$ può essere eliminato
- Con la costruzione a punto fisso a partire da $\langle s_0, t_0 \rangle$ non viene neppure aggiunto

# Formalizzazione descrittiva

## Chiusura di famiglie di linguaggi

- Dati due automi $\mathcal{A}_1 : (\mathbf{Q}_1, \mathbf{I}_1, \delta_1, q, \mathbf{F}_1)$, $\mathcal{A}_2 : (\mathbf{Q}_2, \mathbf{I}_2, \delta_2, s, \mathbf{F}_2)$ l'automa intersezione $\mathcal{A}_\cap = \langle \mathcal{A}_1, \mathcal{A}_2 \rangle$ è dato da
  - Insieme degli stati $\mathbf{Q}_\cap = \mathbf{Q}_1 \times \mathbf{Q}_2$
  - Alfabeto $\mathbf{I}_\cap = \mathbf{I}_1 \cap \mathbf{I}_2$
  - Funzione di transizione $\delta_\cap(\langle q_1, q_2 \rangle, i) = \langle \delta_1(q_1, i), \delta_2(q_2, i) \rangle$
  - Insieme degli stati finali $\mathbf{F}_\cap = \mathbf{F}_1 \times \mathbf{F}_2$

## Correttezza del riconscitore: $L(\mathcal{A}_\cap) = L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$

- Si può dimostrare per induzione sulla lunghezza dell'input

# Unione di linguaggi

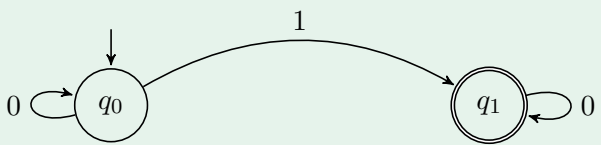
## Due soluzioni

- Una prima soluzione è usare una costruzione analoga all'intersezione, ma che ottiene
  - Insieme degli stati $\mathbf{Q}_\cup = (\mathbf{Q}_1 \times \mathbf{Q}_2) \cup \mathbf{Q}_1 \cup \mathbf{Q}_2$
  - Alfabeto $\mathbf{I}_\cup = \mathbf{I}_1 \cup \mathbf{I}_2$
  - $\delta_\cup(\langle q_1, q_2\rangle, i) = \begin{cases} \langle \delta_1(q_1,i), \delta_2(q_2,i)\rangle \text{ se esistono } \delta_1(q_1,i), \delta_2(q_2,i) \\ \delta_1(q_1,i) \text{ oppure } \delta_2(q_2,i) \text{ altrimenti.} \end{cases}$
  - Insieme degli stati finali $\mathbf{F}_\cup = (\mathbf{F}_1 \times \mathbf{F}_2) \cup \mathbf{F}_1 \cup \mathbf{F}_2$
- La seconda soluzione è applicare la legge di De Morgan: $L_1 \cup L_2 = \neg(\neg L_1 \cap \neg L_2)$ ed usare intersezione e complemento

# Complemento

## Costruito operativamente

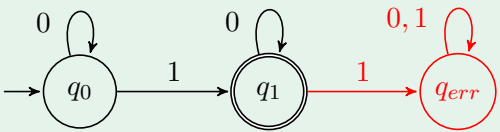
- Idea generale: "rendere finali gli stati non finali e viceversa" ($\mathbf{F}_{compl} = \mathbf{Q} \setminus \mathbf{F}$)
- È sufficiente scambiare i ruoli degli stati?



# Complemento

## Gestire $\delta$ parziali

- No. $\delta$ è una funzione parziale, negli FSA uno stato non accettante è "implicito": lo stato di errore




- Aggiunto $q_{err}$ nell'FSA, "scambiare $\mathbf{F}$ e $\mathbf{Q} \setminus \mathbf{F}$" funziona
- Problema non così facile da risolvere con altri modelli di calcolo
- In generale: calcolare la risposta negativa a un quesito non è equivalente a quella positiva.

# Automi a pila

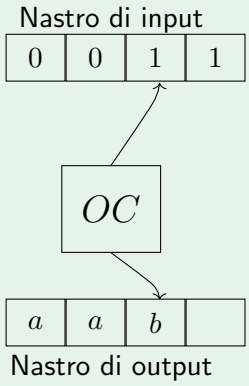
Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

20 febbraio 2025

# Aumentiamo la potenza di un FSA

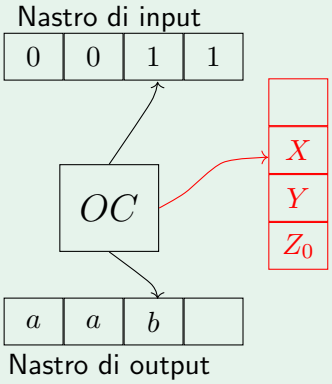
## Descrizione operativa dell'automa a pila

- Un FSA ha un Organo di Controllo (OC) con memoria finita e un nastro di input infinito su cui non può scrivere
- Se traduttore ha un nastro di output in cui può solo scrivere
- La "memoria" dello stato del calcolo è finita



# Aumentiamo la potenza di un FSA

## Una memoria estesa




Aggiungiamo una memoria a impilamento:

- La pila può crescere all'infinito
- Accesso alla sola cella in cima
- Leggere la cima della pila la cancella

$\rightarrow$ Funzionamento Last In First Out (LIFO)

# Automa a pila

## Descrizione operativa

- L'automa a pila compie una mossa in funzione di:
  - Simbolo letto dalla cima della pila
  - Stato corrente nell'FSA che costituisce l'organo di controllo
  - Opzionalmente, simbolo letto dal nastro d'ingresso
- L'automa a pila passa alla configurazione successiva:
  - cambiando stato nell'OC
  - sostituendo al simbolo in cima alla pila una stringa $\alpha$ di simboli (potenzialmente, $\alpha = \varepsilon$)
  - spostando (opzionalmente) la testina di lettura
  - se l'automa è un traduttore, scrivendo una stringa (potenzialmente nulla)

# Riconoscitori e traduttori

## Automa riconoscitore

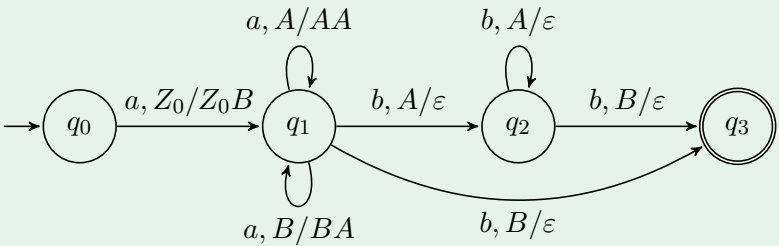
- La stringa $x$ in ingresso è riconosciuta (accettata) se
  - L'automa scandisce completamente $x$
  - Una volta scandita tutta, lo stato dell'OC è di accettazione

## Automa traduttore

- Se la stringa è accettata, il nastro di scrittura contiene la sua traduzione al termine del calcolo $\tau(x)$
- Se la $x$ non è accettata la traduzione è indefinita $\tau(x) = \perp$

# Esempio: Riconoscere $\{a^n b^n | n > 0\}$

## Automa riconoscitore

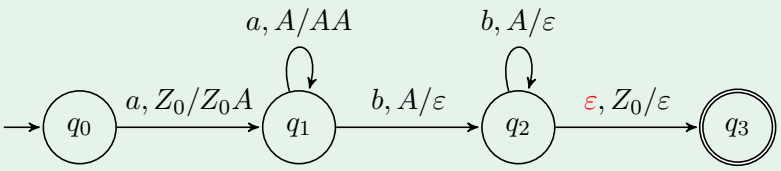



## Convenzioni di notazione

- Etichetta archi: $\langle$lettura input, cima della pila letta/stringa scritta in pila$\rangle$
- Consideriamo la pila inizializzata con $Z_0$ per marcare il fondo

# Esempio: Riconoscere $\{a^n b^n | n > 0\}$

## Un'alternativa

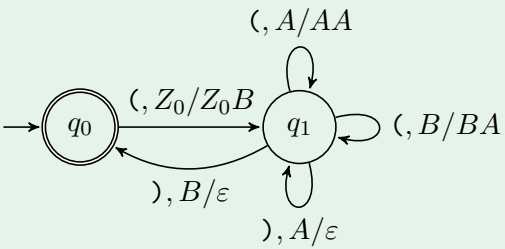



## $\varepsilon - mossa$

- Questo automa effettua una mossa senza leggere dall'input
- Posso evitare di usare $B$ come "marcatore della prima $a$"

# Stringhe ben parentetizzate...

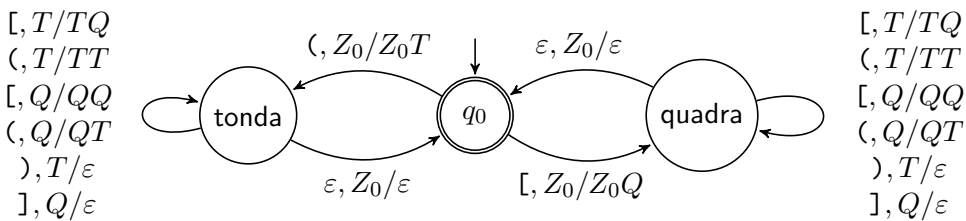
## .. di sole parentesi tonde




## Note

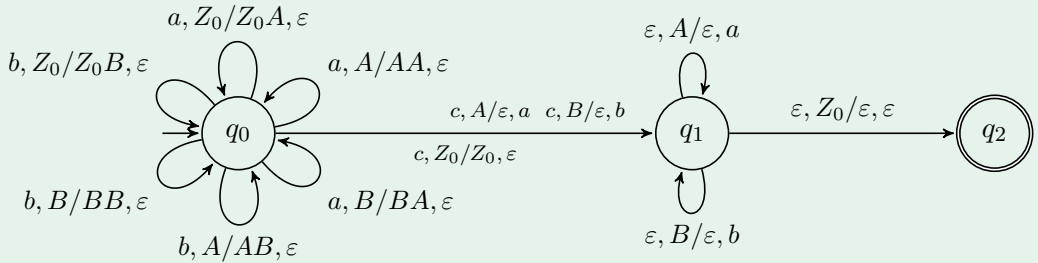
- È una "semplificazione" del riconoscitore di $L = \{a^n b^n\}$
- Verifica solamente che il numero di $a$ coincida con quello di $b$

# Stringhe ben parentetizzate con parentesi tonde e quadre



# Un traduttore

## Da $L_1 \subset \{a, b, c\}^*$ a $L_2 \subset \{a, b, c\}^*$



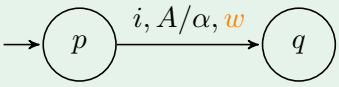

- Che traduzione effettua? (impila $A$ e $B$ fino alla prima $c$ ...)

# Formalizzazione

## Riconoscitore e traduttore

- Automa [traduttore] a Pila : $\langle \mathbf{Q}, \mathbf{I}, \Gamma, \delta, q_0, Z_0, \mathbf{F}[, \mathbf{O}, \eta] \rangle$
- $\mathbf{Q}, \mathbf{I}, \delta, q_0, \mathbf{F}[, \mathbf{O}]$ come nell'FSA [traduttore]
- $\Gamma$ alfabeto di pila (per comodità, disgiunto da $\mathbf{I}$,$[, \mathbf{O}]$)
- $Z_0 \in \Gamma$ simbolo iniziale di pila
- $\delta : \mathbf{Q} \times (\mathbf{I} \cup \{\varepsilon\}) \times \Gamma \to \mathbf{Q} \times \Gamma^*$ (n.b. $\delta$ è parziale)
- $\eta : \mathbf{Q} \times (\mathbf{I} \cup \{\varepsilon\}) \times \Gamma \to \mathbf{O}^*$ ($\eta$ è definita solo dove lo è $\delta$)

## Convenzione grafica




$\delta(p, i, A) = \langle q, \alpha \rangle$
$[\eta(p, i, A) = w]$

# Generalizzare lo stato

## Il concetto di *configurazione*

- Catturare lo stato di un Automa a Pila (AP o PDA, Push-Down Automaton) richiede più informazione di quella di un FSA
- Chiamiamo lo stato di un AP *configurazione* $c = \langle q, x, \gamma, [z] \rangle$
  - $q \in \mathbf{Q}$: stato dell'organo di controllo
  - $x \in \mathbf{I}^*$: stringa ancora da leggere (testina sul 1º carattere di $x$)
  - $\gamma \in \Gamma^*$ stringa dei caratteri in pila; convenzione: la pila cresce da sinistra (basso) a destra (alto)
  - $z \in \mathbf{O}^*$ stringa scritta in output

# Formalizzare la transizione

## Transizione tra configurazioni

- Transizione di un AP: $c \vdash c' : \langle q, x, \gamma, [z] \rangle \vdash \langle q', x', \gamma', [z'] \rangle$
- Per chiarezza abbiamo $\gamma = \beta A$, definiamo, a seconda dei casi:
  1. **Lettura effettiva**: con $x = i.y$ e $\delta(q, i, A) = \langle q', \alpha \rangle$ (definita, non $\perp$) $[\eta(q, i, A) = w]$ abbiamo $x' = y$, $\gamma' = \beta\alpha$, $[z' = z.w]$
  2. $\varepsilon$-**Lettura**: con $x = y$ e $\delta(q, \varepsilon, A) = \langle q', \alpha \rangle$ (definita, non $\perp$) $[\eta(q, \varepsilon, A) = w]$ abbiamo $x' = y$, $\gamma' = \beta\alpha$, $[z' = z.w]$
- Nota bene: $\forall q, A, \delta(q, \varepsilon, A) \neq \perp \Rightarrow \forall i, \delta(q, i, A) = \perp$
- Se ciò non accade, l'AP è *non-deterministico*
  - Approfondiremo questo concetto più avanti nel corso, trattiamo per ora AP deterministici

# Accettazione e traduzione

## Sequenza di mosse

- Definiamo $\overset{*}{\vdash}$ come chiusura riflessiva, transitiva di $\vdash$

## Accettazione e traduzione di $x \in L$

$$x \in L \wedge [z = \tau(x)]$$

$$\Leftrightarrow$$

$$c_0 = \langle q_o, x, Z_0, [\varepsilon] \rangle \overset{*}{\vdash} c_f = \langle q, \varepsilon, \gamma, [z] \rangle, q \in \mathbf{F}$$

N.b. attenzione alle $\varepsilon$ mosse, soprattutto a fine stringa!

# Automi a Pila nella pratica

## Usi degli AP nel software

- Parte fondamentale degli analizzatori sintattici dei compilatori
  - Esistono strumenti sw che generano l'implementazione dell'AP a partire da specifiche sintetiche del linguaggio (corso di Formal Languages and Compilers)
- Macchina astratta che compone l'interprete di Python e Java
- Controllo di correttezza di molti data-description languages, tra cui JSON, BSON, XML, HTML-4

# Proprietà degli AP (riconoscitori)

## Cosa posso riconoscere?

- Un AP è in grado di riconoscere $\{a^n b^n | n > 0\}$, $\{a^n b^{3n} | n > 0\}$
- Posso riconoscere $\{a^n b^n c^n | n > 0\}$?:
  - NO. Intuitivamente: Dopo aver impilato un simbolo per ogni $a$ e spilato uno per ogni $b$, come conto le $c$?
  - Per la dimostrazione formale si usa l'estensione del pumping lemma per i linguaggi riconosciuti dagli AP
  - Pumping lemma per AP: Esiste una costante $k \geq 1$ tale per cui, data $x = pvcws \in L_{AP}, |x| \geq k$ con $|vcw| \leq k$, $|vc| \geq 1 \Leftrightarrow \forall n \in \mathbb{N}, pv^n cw^n s \in L_{AP}$
- La pila è una memoria distruttiva: per leggere occorre cancellare elementi!

# Proprietà degli AP (riconoscitori) - 2

## Cosa posso riconoscere?

- Un AP riconosce sia $\{a^n b^n | n > 0\}$ che $\{a^n b^{2n} | n > 0\}$
- Posso riconoscere $\{a^n b^n | n > 0\} \cup \{a^n b^{2n} | n > 0\}$
  - NO. Intuitivamente "simile" a prima:
  - Se svuoto la pila per contare le prime $n$ $b$ perdo memoria per le successive
  - Se ne svuoto solo metà, e ce ne sono solo $n$ non so se sono a metà pila
  - Intuitivamente: mi servirebbe "dare un'occhiata" in avanti sull'input, per un numero arbitrariamente grande di caratteri
  - Dimostrazione diversa dalla precedente (non banale, diversa dalla precedente, serve il *double-service* lemma)

# Conseguenze delle proprietà

## La famiglia $\mathbf{L}_{AP}$

- $\mathbf{L}_{AP}$: la famiglia di linguaggi riconosciuti dagli AP deterministici
- $\mathbf{L}_{AP}$ non è chiusa rispetto all'unione per quanto detto
- $\mathbf{L}_{AP}$ è chiusa rispetto al complemento? Sì.
  - Il principio della dimostrazione è lo stesso degli FSA, scambiare $\mathbf{F}$ con $\mathbf{Q} \setminus \mathbf{F}$
- $\mathbf{L}_{AP}$ non è chiusa rispetto all'intersezione (perché?)

# Costruire il complemento

## Difficoltà nella costruzione

- La $\delta$ dell'automa va completata come per gli FSA con lo stato di errore
  - Le $\varepsilon$ mosse possono introdurre non-determinismo
- Un ciclo di $\varepsilon$ mosse può evitare che l'automa proceda (stringa non accettata, neppure dall'automa con $\mathbf{F}' = \mathbf{Q} \setminus \mathbf{F}$)
  - Si può trasformare ogni AP con cicli di $\varepsilon$ mosse in uno equivalente privo di essi
- Se esiste una sequenza $\langle q_1, \varepsilon, \gamma_1 \rangle \vdash \langle q_2, \varepsilon, \gamma_2 \rangle \vdash \langle q_3, \varepsilon, \gamma_3 \rangle$ dove solo $q_1, q_3 \in \mathbf{F}$, ma $q_2 \notin \mathbf{F}$ cosa succede?
  - Serve "forzare" l'automa ad accettare solo alla fine di una sequenza (necessariamente finita) di $\varepsilon$ mosse
- *Più della tecnica di dimostrazione è importante: per impiegare la macchina che risolve il "problema positivo" anche per risolvere il "complemento" serve essere sicuri che termini*

# Macchina di Turing

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

28 febbraio 2024

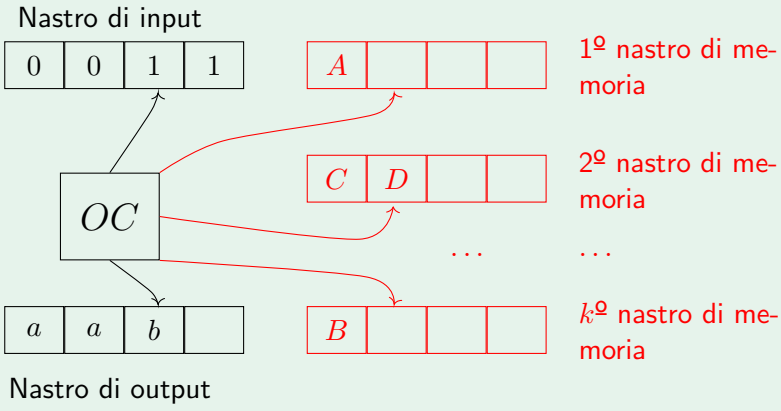
# Un modello di calcolo universale

## Macchina di Turing

- Gli AP sono più potenti degli FSA (= maggiori capacità riconoscitive), ma mostrano ancora limitazioni
- Esaminiamo un modello di calcolo con maggiori capacità
  - Saremo in grado di effettuare "qualunque" calcolo
  - Esamineremo qual é il costo di questa generalità espressiva
- Il modello è la *Macchina di Turing* (MT) (Alan Turing 1912-1954)
  - Uno dei primi modelli di calcolo: sorprendentemente "semplice" nella sua efficacia
  - Esaminiamo ora il funzionamento come riconoscitore e traduttore, in seguito le proprietà universali del calcolo automatico
- Tra le (due) varianti, esaminiamo prima la MT a $k$ nastri per semplicità

# Macchina di Turing

## Modello a $k$-nastri



# Descrizione informale

## Componenti

- Insieme di stati dell'OC, alfabeto $\mathbf{I}$ e $\mathbf{O}$ come per gli AP
- Ogni nastro di memoria può avere un suo alfabeto dedicato o meno: considereremo un unico alfabeto $\Gamma$
- Per convenzione storica e semplicità di formalizzazione, i nastri sono sequenze infinite di celle
  - Solo una quantità finita è inizializzata con un valore sensato
  - Celle restanti contenenti uno spazio vuoto o *blank*: $\textvisiblespace$
- Tutte le testine possono scorrere sui nastri o rimanere ferme
- Configurazione di MT: stato dell'OC e contenuto dei nastri

# Transizione della MT

## In funzione di ...

- Lettura del carattere sotto la testina del nastro di input
- Lettura dei caratteri sotto le testine dei nastro di memoria
- Stato dell'OC

## ... azione conseguente

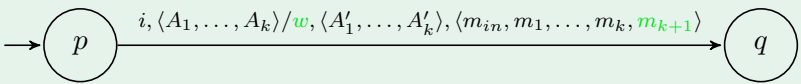
- Cambiamento di stato dell'OC
- Scrittura di un carattere su ogni nastro di memoria
- Scrittura di un carattere sul nastro di uscita
- Spostamento delle testine
  - Testine di memoria e ingresso: 1 posizione a sinistra (`L`), destra (`R`) o ferme (`S`)
  - Testina di output: solo `S` o `R`, se si sposta scrive sempre una lettera o un $\flat$

# Transizione della MT

## Riconoscitore e traduttore

- Transizione $\delta : \mathbf{Q} \times (\mathbf{I} \cup \{\mathfrak{b}\}) \times \Gamma^k \to \mathbf{Q} \times \Gamma^k \times \{\mathtt{L}, \mathtt{S}, \mathtt{R}\}^{k+1}$
- Traduzione $\eta : \mathbf{Q} \times (\mathbf{I} \cup \{\mathfrak{b}\}) \times \Gamma^k \to (\mathbf{O} \cup \{\mathfrak{b}\}) \times \{\mathtt{S}, \mathtt{R}\}$
  - Per quale ragione $\mathbf{O}$ non è meno generale di $\mathbf{O}^*$ ?

## Convenzione grafica




- Mossa $m_{in} \in \{\mathtt{L}, \mathtt{S}, \mathtt{R}\}$: testina di input
- Nastri da $i = 1$ a $i = k \to$ memoria, mosse $m_i \in \{\mathtt{L}, \mathtt{S}, \mathtt{R}\}$
- Nastro $k + 1 \to$ output, $m_{k+1} \in \{\mathtt{S}, \mathtt{R}\}$

# Transizione della MT

## Configurazione iniziale

- Tutti i nastri di memoria pieni di $\mathfrak{b}$, tranne nella posizione iniziale della rispettiva testina dove c'è $Z_0$
- Per farvi riferimento, la posizione delle testine conta da $0$
- Stato iniziale dell'organo di controllo $q_0$
- Stringa in ingresso $x$ scritta a partire dalla $0^{\underline{\mathfrak{a}}}$ cella del nastro di ingresso, seguita e preceduta da $\mathfrak{b}$
- Nastro di output riempito con $\mathfrak{b}$

# Transizione della MT

## Configurazione finale e accettazione

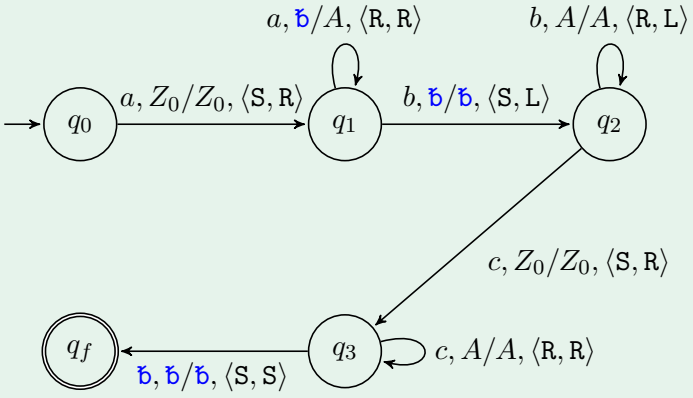
- Stati di accettazione come per FSA, AP: $\mathbf{F} \subseteq \mathbf{Q}$
- Per comodità (convenzione) la $\delta, \eta$ non è definita a partire dagli stati finali : $\forall q \in \mathbf{F},\ \delta(q, \ldots) = \perp,\ \eta(q, \ldots) = \perp$
- La MT si ferma in uno stato $q$ se $\delta(q, \ldots) = \perp$
- La stringa $x$ in ingresso è accettata se e solo se:
  - In numero finito di transizioni, la macchina si ferma in $q \in \mathbf{F}$

## Mancata accettazione

- Una stringa $x$ in ingresso non è accettata se
  - la macchina si ferma in uno stato $\notin \mathbf{F}$
  - *la macchina non si ferma*

# MT: esempi

## Riconoscere $L = \{a^n b^n c^n, n > 0\}$



# Esempio di traduzione: Incremento di un numero decimale

## Schema di funzionamento

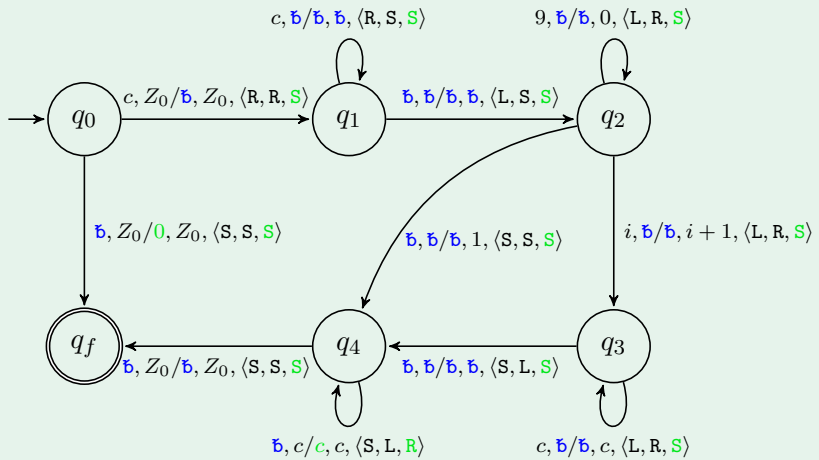
1. Scorri l'input fino alla fine
2. Scorrendo all'indietro l'input, scrivi 0 sul nastro di memoria fino alla prima cifra diversa da 9
3. Scrivi il successore della cifra corrente sul nastro di memoria, dopodichè copia le cifre rimanenti dall'ingresso su di esso
4. Scorrendo al contrario il nastro di memoria, copia le cifre in uscita

## Notazione sintetica

1. `i`: una qualunque cifra tra $0$ e $8$
2. `c`: una qualunque cifra decimale
3. `i+1`: il successore della cifra decimale `i`

# MT: esempi

## Incrementatore decimale (con $\tau(\varepsilon) = 0$)



# Proprietà di chiusura

## Valide

- $\cap$: **OK**, una MT che simula l'esecuzione di due "in sequenza"
- $\cup$: **OK**, una MT che simula l'esecuzione di due "in parallelo"
- . (concatenazione): **OK**, simile a $\cap$
- $*$: **OK** vedi concatenazione

## Non valide

- Complemento: NO. (Dimostrazione nelle prossime lezioni)
  - Se potessi avere MT infinite-loop-free sarebbe facile: individuo l'insieme degli stati di arresto, lo partiziono in finali e non finali, scambio gli insiemi ed è fatta
  - Il problema è nelle *computazioni che non terminano*

# Modelli equivalenti alla MT

## Turing completezza

- È possibile costruire modelli di calcolo equivalenti alla MT
  - Una MT con nastro bidimensionale (mosse {N,S,O,E})
  - Una MT con $k$ testine per nastro
  - Un AP con due pile
- Idea generale: per mostrare che sono equivalenti si mettono in corrispondenza biunivoca le transizioni di una MT "classica" a quelle del modello di calcolo proposto
- Se un modello di calcolo può simulare una MT, viene detto *Turing completo*
  - Quasi tutti i linguaggi di programmazione [a]
  - Il meccanismo di risoluzione dei page fault dell'MMU x86_64[a]

[a] assumendo memoria infinita

# MT a nastro singolo

## Nastro singolo $\neq$ 1 nastro di memoria

- Una particolare variante della MT è quella a nastro singolo: input, output e memoria sono tutti sullo stesso nastro
  - È il modello inizialmente ideato da Turing
- È equivalente (=può emulare) una MT a $k$ nastri
  - La posizione delle testine è tenuta usando un carattere speciale sul nastro, e memorizzando il carattere sotto la testina nell'OC
  - Il contenuto dei diversi nastri è delimitato da altri caratteri speciali, si sequenzializzano le operazioni su nastri diversi

# MT e modelli di calcolo classico

## Differenze con modelli più "realistici"

- Una MT è in grado di simulare il comportamento di una macchina di von Neumann (opportunamente "astratta", con memoria infinita)
- La differenza principale è la modalità di accesso alla memoria
  - Una macchina di von Neumann accede per indirizzo ai dati, mentre una MT scorre il nastro opportuno
  - Nessun cambio a livello di capacità computazionali (= problemi risolvibili)
- Spesso cambiare modello di calcolo ha un significativo impatto in complessità (= tempo/spazio impiegati) del calcolo

# Non Determinismo

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

13 marzo 2024

# Modelli operazionali non deterministici

## Modelli deterministici vs. modelli non deterministici

- Solitamente, un algoritmo è una *sequenza deterministica* di passi
  - Data una configurazionee un input, il passo successivo è unico
- È sempre utile? È sempre il modello più maneggevole?
- Non sempre l'ordine di esecuzione è fondamentale
  - Es. Trovare due calzini dello stesso colore e metterli
  - È importante che la computazione termini con due calzini dello stesso colore addosso, non "come ci sono arrivati"

# Modelli operazionali non deterministici

## La “comodità” del non determinismo

- Una descrizione di un algoritmo non deterministico è solitamente più compatta (sacrificando operatività)

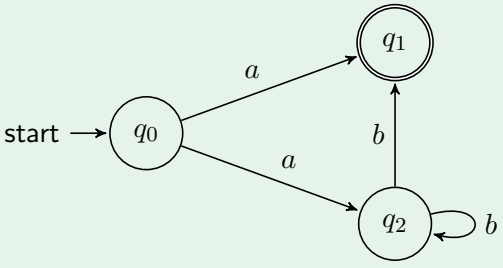
Non-det. “Trova una strada dall’entrata all’uscita”

Det. `https://en.wikipedia.org/wiki/Maze_solving_algorithm`

- Per una realizzazione pratica si possono sfruttare più esecutori
- Simmetricamente, un modello di esecuzione non-deterministico può essere utile come semantica per il calcolo parallelo (pthreads, Ada, OpenCL inter workgroups)

# Versioni nondeterministiche (ND) di modelli noti

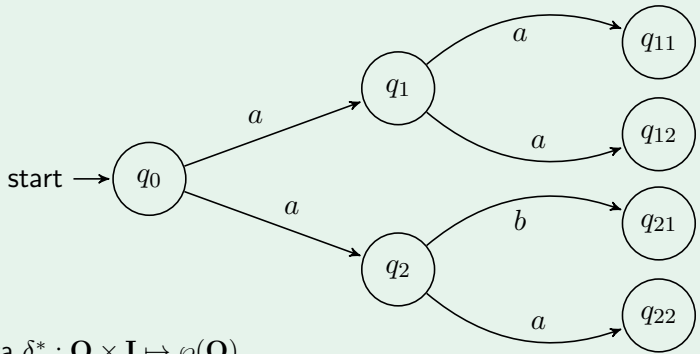
## Automi a stati finiti ND (riconosce $L = ab^*$)




- La $\delta$ diventa $\delta : \mathbf{Q} \times \mathbf{I} \mapsto \wp(\mathbf{Q})$
- Nell'esempio abbiamo $\delta(q_0, a) = \{q_1, q_2\};\ \delta(q_2, b) = \{q_1, q_2\}$

# Automi a stati finiti ND

## Formalizzazione della sequenza di mosse




- La $\delta$ diventa $\delta^* : \mathbf{Q} \times \mathbf{I} \mapsto \wp(\mathbf{Q})$
- Nell'esempio abbiamo $\delta(q_0, a) = \{q_1, q_2\};\ \delta(q_2, b) = \{q_{21}\}$
- Di conseguenza, $\delta^*(q_0, aa) = \delta(q_1, a) \cup \delta(q_2, a) = \{q_{11}, q_{12}, q_{22}\}$

# Automi a stati finiti ND

## Formalizzazione della sequenza di mosse - 2

- Estensione di $\delta$ a stringhe:

$$\delta^*(q,x) = \begin{cases} \delta(q,\varepsilon) = \{q\}, \text{ con } x = \varepsilon \\ \delta(q,y.i) = \bigcup_{r \in \delta^*(q,y)} \delta(r,i), \text{ con } x = y.i, i \in \mathbf{I} \end{cases}$$

## Accettazione di un FSA-ND

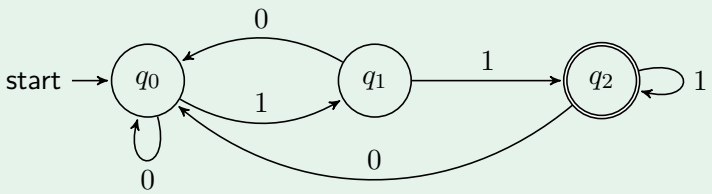
- Nostra convenzione: l'FSA-ND accetta $x$ se almeno uno degli stati dell'insieme $\delta^*(q_0,x)$ è finale, ovvero

$$x \in L \Leftrightarrow (\delta^*(q_0,x) \cap \mathbf{F}) \neq \varnothing$$

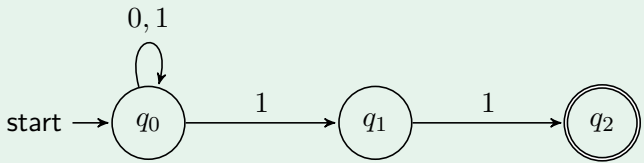
- È possibile anche considerare $\delta^*(q_0,x) \subseteq \mathbf{F}$

# Automi a stati finiti ND e D a confronto

## FSA-D per $L = \{0,1\}^*11$ (stringhe che terminano in 11)

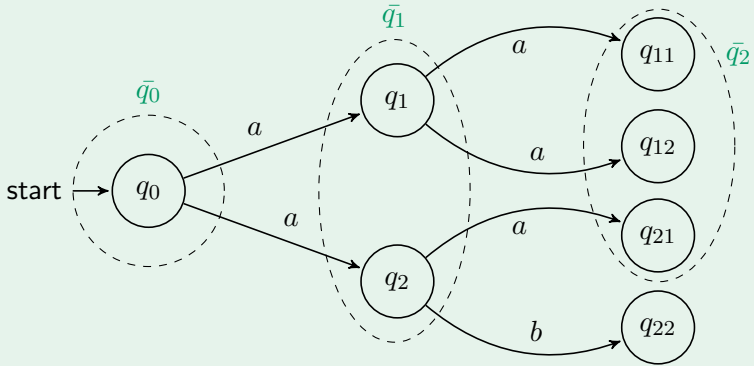



## FSA-ND per $L = \{0,1\}^*11$ (stringhe che terminano in 11)



# Automi a stati finiti ND e D a confronto

## FSA ND ha maggior potere riconoscitivo di FSA? No...




- Intuitivamente: Otteniamo un FSA-D equivalente chiamando “stato” ogni insieme d’arrivo della $\delta$ e mantenendo gli archi

# Automi a stati finiti ND e D a confronto

## Sistematizzando il confronto

- Posso sempre costruire *automaticamente* un FSA-D equivalente a un ND dato:
- Da $\mathcal{A}_{nd} = \langle \mathbf{Q}_n, \mathbf{I}_n, \delta_n, q_{0n}, \mathbf{F}_n \rangle$ ricavo $\mathcal{A}_d = \langle \mathbf{Q}_d, \mathbf{I}_d, \delta_d, q_{0d}, \mathbf{F}_d \rangle$
  - $\mathbf{Q}_d = \wp(\mathbf{Q}_n)$, $\mathbf{I}_d = \mathbf{I}_n$
  - $\delta_d(q_d, i) = \bigcup_{q_n \in q_d} \delta_n(q_n, i)$
  - $q_{d0} = \{q_{n0}\}$
  - $\mathbf{F}_d = \{s \in \wp(\mathbf{Q}_n) | s \cap \mathbf{F}_n \neq \varnothing\}$
- Gli FSA-ND *non sono* più potenti degli FSA-D

# Usi degli FSA-ND

## Comodità di specifica

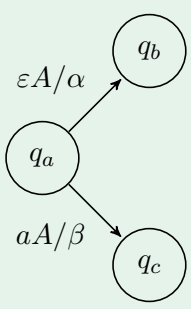
- Se FSA-D e FSA-ND sono equivalenti come potere riconoscitore, perchè mantenere gli ND (più "problematici")?
- Può essere più comodo specificare un FSA ND e poi far ricavare automaticamente il corrispondente D ad un programma, risparmiando la fatica di concepire quello D
- Attenzione: la determinizzazione di un FSA ha un costo: alla peggio $|\mathbf{Q}_d| = |\wp(\mathbf{Q}_n)| = 2^{|\mathbf{Q}_n|}$
  - É il caso pessimo, non tutti gli stati sono necessariamente raggiungibili, ma può succedere
  - Esempio: l'FSA riconoscitore di stringhe con un dato suffisso

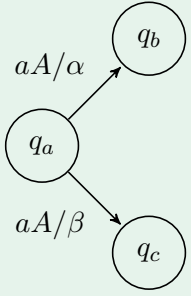
# Automi a pila non deterministici

## Come ottenerli?

- "Naturalmente" non deterministici, basta eliminare la restrizione imposta fino ad ora

## Transizioni ammesse per AP-ND



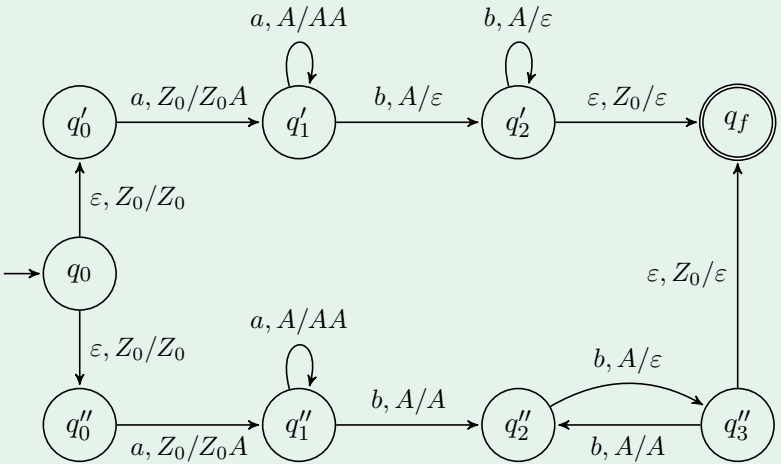
# Automi a pila non deterministici

## Effetti della generalizzazione

- Otteniamo la $\delta_{\text{AP-ND}} : \mathbf{Q} \times (\mathbf{I} \cup \{\varepsilon\}) \times \Gamma \to \wp_F(\mathbf{Q} \times \Gamma^*)$
  - Come mai il pedice $F$? I possibili sottoinsiemi di $\mathbf{Q} \times \Gamma^*$ sono infiniti, consideriamo solo quelli ottenibili dalle coppie nell'immagine di $\delta_{\text{AP-ND}}$, ottenendo un insieme delle parti *finito*
- L'AP ND accetta $x$ se esiste una sequenza $c_o \overset{*}{\vdash} \{c_1, \ldots c_n\}$ con $c_0 = \langle q_0, x, Z_0 \rangle, c_1 = \langle q, \varepsilon, \gamma \rangle, q \in \mathbf{F}$
- N.B. $\vdash$ non è più univoca!

# Automi a pila non deterministici

## Un esempio di riconoscitore



# Conseguenze del riconoscitore d'esempio

## Proprietà degli AP ND

- L'automa precedente è in grado di riconoscere $\{a^n b^n\} \cup \{a^n b^{2n}\}$
  - Gli AP ND sono più potenti degli AP D
- Generalizzando l'AP precedente si ottiene una dimostrazione *costruttiva* della chiusura di AP ND rispetto all'unione
  - Proprietà *non* condivisa dagli AP D
- Gli AP ND non sono chiusi rispetto all'intersezione
  - $\{a^n b^n c^*\} \cap \{a^* b^n c^n\} = \{a^n b^n c^n\}$ non è riconoscibile neppure in modo ND da un AP (il pumping lemma per AP vale anche per quelli ND)

# Conseguenze delle proprietà

## Chiusura rispetto al complemento

- Se la famiglia di linguaggi $\mathbb{L}_{\mathsf{APND}}$ è chiusa rispetto a $\cup$ e non rispetto a $\cap$ non può esserlo rispetto al complemento
- Il nondeterminismo impatta direttamente sulla complementazione:
  - Se un automa è deterministico e termina sempre la sua computazione è sufficiente scambiare accettazione e non accettazione per avere il complemento
- Con gli APND la computazione termina sempre ma se ho:
  - $\langle q_0, x, Z_0\rangle = c_0 \overset{*}{\vdash} \{\langle q_1, \varepsilon, \gamma_1\rangle, \langle q_2, \varepsilon, \gamma_2\rangle\},\ q_1 \in \mathbf{F}, q_2 \notin \mathbf{F}$
  - $x$ è accettata nella computazione precedente, ma continua ad esserlo anche se scambio $\mathbf{F}$ con $\mathbf{Q} \setminus \mathbf{F}$

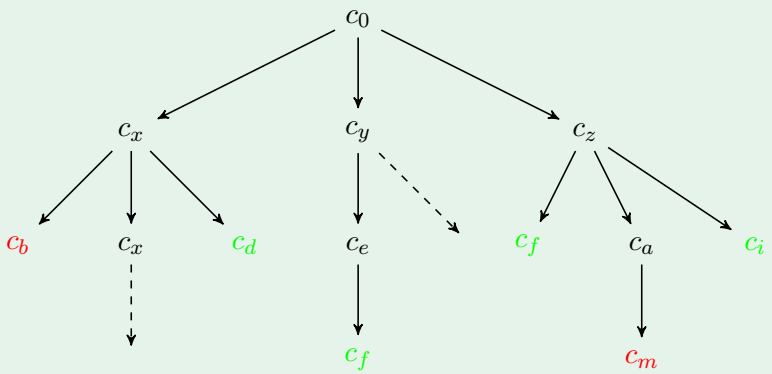
# Macchine di Turing nondeterministiche

## Definizione e caratteristiche

- $\delta : \mathbf{Q} \times \mathbf{I} \times \Gamma^k \to \wp(\mathbf{Q} \times \Gamma^k \times \{\mathtt{L}, \mathtt{S}, \mathtt{R}\})$ (perchè $\wp$ e non $\wp_F$?)
- Configurazione, transizione, sequenza di transizioni e accettazione definite come negli altri casi
- Prima domanda: le MT ND sono più potenti delle MT D?

# Albero delle computazioni

## Rappresentare una computazione ND




Verde: Computazione accettante, Rosso: Computazione non accettante

# Equivalenza tra MT ND e MT D

## Emulare una MT ND con una MT D

- $x$ è accettata da una MT ND solo se esiste un calcolo che termina in uno stato di accettazione
- Come emulare una MT ND con una D?
  - Percorrere l'albero delle computazioni ND per stabilire se esiste un percorso che termina in uno stato di accettazione
  - Nel caso di un albero "normale", esistono algoritmi consolidati per effettuare questa visita
  - Come gestisco le computazioni che non terminano?
- Visita dell'albero "in ampiezza"
  - Costruisco una MT D che scandisce le configurazioni della ND a partire dalle più vicine a $c_0$
  - Intuitivamente: se la MT ND termina, termina anche la mia MT D con lo stesso esito

# Conclusioni sul nondeterminismo

## Un utile formalismo

- Utile per rappresentare problemi/algoritmi dove alcune scelte locali non sono fattibili al momento/importanti
- Aumenta la potenza dei soli AP (tra i formalismi visti)
- Può essere applicato praticamente a tutti i modelli di calcolo (estensione facile ai traduttori)
- N.B.: nondeterministico $\neq$ probabilistico
  - La computazione procede sempre con *certezza* verso l'insieme di stati successivo
  - Esistono modelli di calcolo probabilistico, ma sono ben diversi (e.g., FSA-prob $\approx$ catene di Markov)

# Grammatiche

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

20 marzo 2024

# Modelli Generativi

## Grammatiche

- I modelli di linguaggio/calcolo visti finora definiscono un linguaggio tramite l'*elaborazione* della stringa che gli appartiene
- Vediamo un modello *generativo* del linguaggio: la grammatica
- In generale, una *grammatica* o *sintassi* è un insieme di regole per generare *frasi* di un linguaggio
- Modello molto antico (Pāṇini ne fornì una per il sanscrito tra il 6º e il 4º secolo a.C.), ma di grande efficacia

# Grammatica come sistema di riscritture

## Esempi (informali)

- "Una frase è formata da un soggetto seguito da un predicato"
  - "Un soggetto è un sostantivo, o un pronome, oppure ..."
  - "Un predicato è un verbo seguito da complementi, oppure ..."
  - Frase $\rightarrow$ soggetto.predicato $\rightarrow$ "Pierino mangia la mela"
  - Specifica sintattica: vale anche "La mela mangia Pierino"
- "Una funzione `ANSI-C-89` è composta da un prototipo, una parte dichiarativa, una esecutiva"
  - Programma $\rightarrow$ prototipo.dichiarativa.esecutiva $\rightarrow$ `int f(void) {int a; a=a+1; return a;}`
- "Un cartello di segnaletica verticale è di obbligo, divieto, pericolo, precedenza,..."
  - "Un cartello di pericolo è di forma triangolare, bordato in rosso e contiene..."

# Grammatica come sistema di riscritture

## Riscritture per raffinamenti successivi

- Le regole di una grammatica descrivono un "oggetto principale" (libro, protocollo, messaggio grafico) come un insieme ordinato di "componenti"
- La descrizione è fornita fino ad arrivare al livello di dettaglio desiderato (carattere, bit, forma geometrica elementare)
- Ogni passo di riscrittura può offrire una o più alternative
  - Il soggetto può essere un nome, un pronome ...
- Spesso si tende a chiamare *lessico* la descrizione grammaticale delle singole "parole", *sintassi* quella della loro composizione
  - Dal nostro punto di vista, è un riuso dello stesso modello

# Grammatica

## Definizione formale

- Una grammatica è una quadrupla $G = \langle \mathbf{V}_t, \mathbf{V}_n, \mathbf{P}, S \rangle$
  - $\mathbf{V}_t$: alfabeto o vocabolario *terminale*
  - $\mathbf{V}_n$: alfabeto o vocabolario *nonterminale*
  - $\mathbf{V} = \mathbf{V}_n \cup \mathbf{V}_t$: alfabeto o vocabolario
  - $S \in \mathbf{V}_n$: elemento di $\mathbf{V}_n$ detto *assioma* o *simbolo iniziale*
  - $\mathbf{P} \subseteq \mathbf{V}_n^+ \times \mathbf{V}^*$ insieme delle produzioni sintattiche o regole di riscrittura
- Per semplicità di notazione, indicheremo gli elementi $p \in \mathbf{P}$, $p = \langle \alpha, \beta \rangle$ come $p = \alpha \to \beta$

# Grammatica

## La relazione di derivazione

- Definiamo la relazione di derivazione immediata $\underset{G}{\Rightarrow}$ per una grammatica $G = \langle \mathbf{V}_t, \mathbf{V}_n, \mathbf{P}, S \rangle$ come $\alpha \underset{G}{\Rightarrow} \beta$ se e solo se $\alpha \in V^+, \beta \in V^*, \alpha = \alpha_1\alpha_2\alpha_3, \beta = \alpha_1\beta_2\alpha_3,\ \alpha_2 \to \beta_2 \in \mathbf{P}$
- Esempio: Rispetto a $G = \langle \{ab\}, \{S, R\}, \{S \to aR, R \to bS, S \to \varepsilon\}, S \rangle$, abbiamo che $abaR \underset{G}{\Rightarrow} ababS$ ma non è vero che $abaR \underset{G}{\Rightarrow} aba$
- Dove non ambiguo, ometteremo il pedice $G$ che indica la grammatica
- Definiamo $\overset{*}{\Rightarrow}$ come la chiusura riflessiva e transitiva di $\Rightarrow$

## Linguaggio generato da una grammatica

Il linguaggio $L(G)$ generato dalla grammatica $G$ è l' insieme di tutte e sole le stringhe $x$ di soli caratteri di $\mathbf{V}_t$ tali che $S \overset{*}{\Rightarrow} x$.

# Alcuni esempi - 1

## Una prima grammatica semplice $G_1$

- $\mathbf{V}_t = \{a, b, c\},\ \mathbf{V}_n = \{S, A, B, C\}$ con assioma $S$
- $\mathbf{P} = \{S \to A,\ A \to aA,\ A \to B,\ B \to bB,\ B \to C,\ C \to cC,\ C \to \varepsilon\}$
- Una possibile derivazione è
  $S \Rightarrow A \Rightarrow aA \Rightarrow aaA \Rightarrow aaB \Rightarrow aaC \Rightarrow aacC \Rightarrow aaccC \Rightarrow aacccC \Rightarrow aaccc$
- Un'altra possibile derivazione è $S \Rightarrow A \Rightarrow B \Rightarrow bB \Rightarrow bC \Rightarrow b$
- Linguaggio generato da $G$, $L(G) = \{a^*b^*c^*\}$

# Alcuni esempi - 2

## Qualcosa di più sostanzioso $G_2$

- $\mathbf{V}_t = \{a, b\}, \mathbf{V}_n = \{S\}$ assioma $S$
- $\mathbf{P} = \{S \to aSbS, S \to \varepsilon\}$
- Una possibile derivazione $S \Rightarrow aSbS \Rightarrow aSb \Rightarrow aaSbSb \Rightarrow aaSbb \Rightarrow aabb$
- Una ulteriore possibile derivazione
  $S \Rightarrow aSbS \Rightarrow aSbaSbS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab$
- Linguaggio generato dalla grammatica? Coppie di $a$ e $b$ "ben parentetizzate"

# Alcuni esempi - 3

## Qualcosa di *ancora* più sostanzioso $G_3$

- $\mathbf{V}_t = \{a, b, c\}, \mathbf{V}_n = \{S, A, B, C, D\}$ assioma $S$
- $\mathbf{P} = \{S \to aACD,\ A \to aAC,\ A \to \varepsilon,\ B \to b, CD \to BDc, CB \to BC, D \to \varepsilon\}$
- Una possibile derivazione:
  $S \Rightarrow aACD \Rightarrow aaACCD \Rightarrow aaaACCCD \Rightarrow aaaACCCD \Rightarrow aaaCCCD \Rightarrow aaaCCBDc \Rightarrow aaaCCBDc \Rightarrow aaaCBCDc \Rightarrow aaaBCCDc \Rightarrow aaaBCBDcc \Rightarrow aaaBBCDcc \Rightarrow aaaBBBDccc \Rightarrow aaaBBBccc \overset{*}{\Rightarrow} aaabbbccc$
- Linguaggio generato: $L(G) = \{a^n b^n c^n\}$

# Alcune domande “naturali”

## Utilità pratica

- Dove è possibile utilizzare grammatiche in pratica?

## Espressività

- Quali linguaggi è possibile esprimere con una data grammatica?

## Relazione con i riconoscitori

- Che relazione sussiste tra i linguaggi generati dalle grammatiche e i linguaggi riconosciuti dagli automi?

# Usi pratici delle grammatiche

## Modello descrittivo

- Le grammatiche sono ampiamente usate come modello descrittivo di linguaggi di programmazione (C, Scheme, Pascal, ...) e descrizione dati (JSON)
  - Esiste per alcune di esse la possibilità di ottenere automaticamente l'automa riconoscitore del linguaggio generato

## Modello generativo

- Generazione automatizzata di input di test per programmi
- Sintesi di frasi in linguaggio "naturale"

# Espressività delle grammatiche

## Quali linguaggi è possibile esprimere

- Negli esempi precedenti abbiamo visto come sia possibile generare linguaggi che sappiamo essere riconosciuti, usando un automa a potenza minima
  - Da un FSA: è facile costruire un FSA det. a 3 stati che riconosce $L(G_1)$
  - Da un PDA: il riconoscitore di $L(G_2)$ è stato un esempio che abbiamo visto durante le precedenti lezioni
  - Da una MT: serve una MT per riconoscere $L(G_3) = a^n b^n c^n$
- É possibile classificare le grammatiche in base al loro potere generativo
  - ovvero in base alla famiglia di appartenenza del linguaggio generato

# Espressività delle grammatiche

## Quali linguaggi è possibile esprimere? La gerarchia di Chomsky

- 4 classi, a seconda delle limitazioni (crescenti) imposte sulle produzioni $\alpha \to \beta$

| Tipo | Nome | Limitazione Produzioni |
|---|---|---|
| 0 | Non limitate | nessuna |
| 1 | Monotone (non-decreasing) | $\lvert\alpha\rvert \leq \lvert\beta\rvert$ |
| 1 | Dipendenti dal contesto | $\alpha = \gamma A \delta \quad \beta = \gamma \chi \delta, \chi \neq \varepsilon$ o $S \to \varepsilon$ |
| 2 | Libere dal contesto | $\lvert\alpha\rvert = 1$ |
| 3 | Regolari | di forma $A \to a, A \to aA$, oppure $A \to a, A \to Aa$ con $a \in \mathbf{V}_t, A \in \mathbf{V}_n$ |

- Una grammatica più potente genera tutti i linguaggi di una meno potente. Domanda: l'inclusione è stretta?

# A quali automi corrispondono?

## Grammatiche Regolari e FSA

- Linguaggi generati da grammatiche regolari $\equiv$ riconosciuti da FSA

## Dall'FSA $\mathcal{A}$ alla grammatica

- Poniamo $\mathbf{V}_n = \mathbf{Q}, \mathbf{V}_t = \mathbf{I}, S = \langle q_0 \rangle$
- Per ogni $\delta(q, i) = q'$ aggiungiamo $\langle q \rangle \to i\langle q' \rangle$ all'insieme $\mathbf{P}$
- Se $q' \in \mathbf{F}$ per una data $\delta(q, i) = q'$, aggiungiamo anche $\langle q \rangle \to i$ all'insieme $\mathbf{P}$
- Facile mostrare per induzione che $\delta^*(q_0, x) = q'$ sse $\langle q_0 \rangle \stackrel{*}{\Rightarrow} x\langle q' \rangle$

## Dalla grammatica all'FSA (non deterministico)

- $\mathbf{Q} = \mathbf{V}_n \cup \{q_f\},\ \mathbf{I} = \mathbf{V}_t,\ q_0 = S,\ \mathbf{F} = \{q_f\}$
- Se $A \to bC \in \mathbf{P}$, $\delta(A, b) = C$; Se $A \to b \in \mathbf{P}$, $\delta(A, b) = q_f$

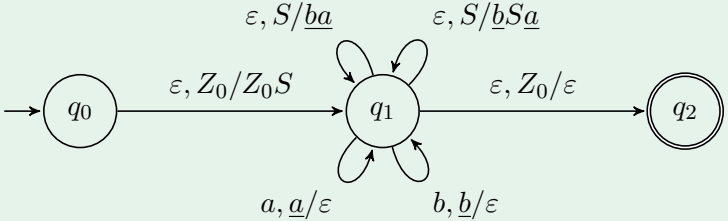
# A quali automi corrispondono?

## Grammatiche Libere dal Contesto e AP-ND

- I linguaggi generati dalle grammatiche libere dal contesto coincidono con i riconosciuti dagli AP non deterministici
- Dimostrazione non banale; intuizione: salvo in pila la coda della forma di frase

## Dalla grammatica all'AP ND: un esempio illustrativo

- Data $\langle \{S\}, \{a, b\}, \{S \to aSb, S \to ab\}, S \rangle$



# A quali automi corrispondono?

## Grammatiche non limitate e MT

- Le grammatiche non limitate (tipo 0) corrispondono alle MT
- Costruiamo, senza pretesa di formalizzazione qui, una MT ND che accetti $L(G)$
- $\mathcal{M}$ ha un nastro di memoria, inizializzato con $Z_0 S$
- La stringa da riconoscere è sul nastro di ingresso
- Il nastro di memoria viene scandito alla ricerca di una parte sinistra di una qualche produzione $p \in \mathbf{P}$
- Quando una viene trovata (scelta nondeterministicamente), viene sostituita con la sua parte destra
- Se ve n'è più di una, si opera ancora nondeterministicamente

# Grammatiche non limitate e MT

## Funzionamento della MT

- Per come abbiamo costruito la MT ND, sappiamo che

$$\alpha \Rightarrow \beta \text{ se e solo se } c = \langle q, Z_0, \alpha \rangle \overset{*}{\vdash} \langle q, Z_0, \beta \rangle$$

- In questo modo, quando (e *se*) sul nastro di memoria si raggiunge un contenuto fatto di soli elementi di $\mathbf{V}_t$, lo si può confrontare con la stringa in ingresso $x$ e
  - se coincide accettare $x$
  - se non coincide, questa computazione tra quelle eseguite nondeterministicamente non è di accettazione
- N.B. Il nondeterminismo della MT è utile, ma non essenziale
- Resta il problema di sapere *se* la MT termina...

# Grammatiche non limitate e MT

## Emulare una MT con una grammatica non ristretta

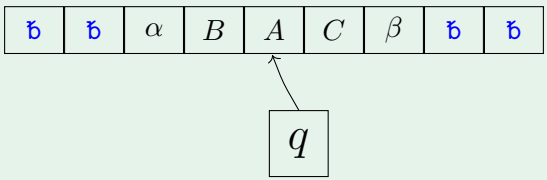
- Senza perdere di generalità, emuliamo una MT $\mathcal{M}$ a nastro singolo con una grammatica $G$ non ristretta
- Considerato che $G$ può "manipolare" solo elementi di $\mathbf{V}_n$, faccio in modo che generi stringhe della forma $x\blacklozenge X$ con $\blacklozenge \in \mathbf{V}_n, x \in \mathbf{V}_t^*$ e $X$ che è costituita da "copie nonterminali" degli elementi di $X$
  - Ad esempio, se $x = abac$, genero la stringa $abac\blacklozenge ABAC$
- Obiettivo: avere una derivazione $x\blacklozenge X \stackrel{*}{\Rightarrow} x$ se e solo se $x$ è accettata da $\mathcal{M}$
- Simuleremo ogni mossa di $\mathcal{M}$ con una derivazione diretta di $G$

## Emulazione della impostazione del nastro con parte iniziale di $G$

- Assumendo $\mathbf{I} = \mathbf{V}_t = \{a, b\}$ la parte delle produzioni di $G$ che genera le stringhe appena descritte è:
  - $S \to SA'A,\ S \to SB'B,\ S \to \blacklozenge$ (genero coppie di simboli)
  - $AA' \to A'A,\ BA' \to A'B$ (faccio "scorrere" le $A'$ a sx)
  - $AB' \to B'A,\ BB' \to B'B$ (faccio "scorrere" le $B'$ a sx)
  - $\blacklozenge A' \to a\blacklozenge,\ \blacklozenge B' \to b\blacklozenge$ (quando "scorro" attraverso $\blacklozenge$ trasformo il nonterminale in terminale)

# Grammatiche non limitate e MT

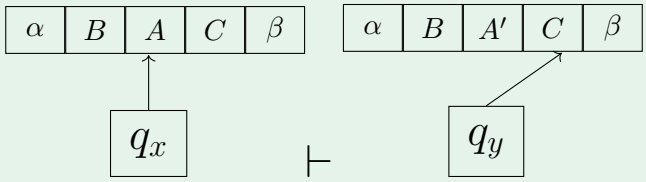
## Emulare le mosse




- Rappresento la configurazione qui sopra con $x\blacklozenge\alpha BQAC\beta$ con $x \in \mathbf{V}_t*$
- Se è definita:
  - $\delta(q, A) = \langle q', A', R\rangle$ aggiungo $QA \to A'Q'$ alle prod. di $G$
  - $\delta(q, A) = \langle q', A', S\rangle$ aggiungo $QA \to Q'A'$ alle prod. di $G$
  - $\delta(q, A) = \langle q', A', L\rangle$ aggiungo, $\forall B$ nell'alfabeto di $\mathcal{M}$, $BQA \to Q'BA'$ alle prod. di $G$ (n.b. l'alfabeto di $\mathcal{M}$ è unico)

# Grammatiche non limitate e MT

## Emulare le mosse




- se e solo se $\blacklozenge\alpha BQ_xAC\beta \Rightarrow \blacklozenge\alpha BA'Q_yC\beta$
- La costruzione di $G$ va completata aggiungendo regole che cancellano tutto ciò che sta a destra del $\blacklozenge$ ($\blacklozenge$ incluso) se e solo se la configurazione della $\mathcal{M}$ è accettante, e.g. $\blacklozenge\alpha BQ_fAC\beta$

# Rimanenti corrispondenze

## Automi a pila deterministici

- Esiste un sottoinsieme (proprio) delle grammatiche libere dal contesto che genera i linguaggi riconosciuti dagli AP deterministici
- Restrizione difficile da esprimere sulla forma delle produzioni
- Dettagliata nel corso di Formal Languages and Compilers

## Grammatiche dipendenti dal contesto

- Le grammatiche di tipo 1 corrispondono a un sottoinsieme delle MT di cui è certo che terminino sempre
- N.B. non si tratta delle uniche MT che terminano sempre
- Consentono sempre di sapere se una stringa $x$ è generata da $G$ (si può costruire l'insieme delle stringhe gen. da $G$ lunghe quanto $x$ e vedere se essa appare)

# Logica Matematica

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

4 aprile 2024

# La logica come formalismo descrittivo

## Un ulteriore linguaggio di specifica

- Logica: un formalismo "universale" alternativo al linguaggio naturale
  - Vantaggi: non ambiguità, possibile dimostrare in modo automatico proprietà desiderate
- Applicata in contesti molto vari: da ingegneria informatica a ingegneria dei sistemi
- Esistono formalismi applicativi basati sulla logica:
  - Linguaggi di programmazione (Prolog, Datalog)
  - Linguaggi di specifica (Z è lo standard ISO/IEC 13568 per specifiche funzionali)

## Usi esaminati in questo corso

- Specifica di linguaggi formali (logica monadica del primo e secondo ordine)
- Logica per la specifica di comportamento (I/O) di programmi

# Logica del prim'ordine su parole finite

## Sintassi ed interpretazione

- Consideriamo la logica del prim'ordine con predicati su una sola variabile. Consente di descrivere parole su un alfabeto $\mathbf{I}$
- Con $a \in \mathbf{I}$ una lettera predicativa per ogni simbolo di $\mathbf{I}$, una formula $\varphi$ è :
  - $a(x)$: il predicato $a$ applicato ad una variabile
  - $x < y$
  - $\neg\varphi$: negazione logica della formula
  - $\varphi \wedge \varphi$: congiunzione (and Booleano) di due formule
  - $\forall x\ (\varphi)$: quantificatore universale
- Interpretazione:
  - il dominio delle variabili è un sottoinsieme finito di $\mathbb{N}$ da pensare come posizioni
  - $<$ corrisponde alla relazione di minore tra le posizioni

# Alcune abbreviazioni

## Concetti ricorrenti

- Come sempre:
  - $\varphi_1 \vee \varphi_2 \stackrel{\Delta}{=} \neg(\neg\varphi_1 \wedge \neg\varphi_2)$: legge di De Morgan
  - $\varphi_1 \Rightarrow \varphi_2 \stackrel{\Delta}{=} \neg\varphi_1 \vee \varphi_2$: definizione di implicazione semplice
  - $\exists x(\varphi) \stackrel{\Delta}{=} \neg\forall x(\neg\varphi)$: proprietà dei quantificatori
  - $x = y \stackrel{\Delta}{=} \neg(x < y) \wedge \neg(x > y)$: definizione di uguaglianza
  - $x \leq y \stackrel{\Delta}{=} \neg(y < x)$
- In aggiunta:
  - La costante $0$: $x = 0 \stackrel{\Delta}{=} \forall y\ (\neg(y < x))$
  - La funzione successore $S(x)$: $S(x) = y \stackrel{\Delta}{=} (x < y) \ \wedge \ \neg\exists z(x < z \wedge z < y)$
  - Le costanti $1, 2, 3, \ldots$ come $S(0), S(S(0)), S(S(S(0))), \ldots$

# Interpretazione come parole su $\mathbf{I}$

## Interpretazione di $a(x)$

- $a(x)$ è vero $\Leftrightarrow$ l'$x$-esimo simbolo di una parola $w \in \mathbf{I}^*$ è $a$
  - gli indici di $w$ partono da $0$

## Esempi

- Formula vera su tutte e sole le parole non vuote che iniziano per $a$:
  $\exists x(x = 0 \wedge a(x))$
- Formula vera su tutte e sole le parole in cui ogni $a$ è seguita da una $b$:
  $\forall x(\ a(x)\ \Rightarrow\ \exists y(y = S(x) \wedge b(y))\ )$
- Formula vera per la sola stringa vuota: $\forall x\ (a(x) \wedge \neg a(x))$
  - n.b. se problematico considerare $\varepsilon$, la formula non è mai vera

# Altre abbreviazioni convenienti

## Abbreviazioni per indici comodi

- $y = x + 1$ indica $y = S(x)$ (non esiste un'operatore di somma tra variabili)
- generalizzando, se $k \in \mathbb{N}, k > 1$ indichiamo con $y = x + k$
  $\exists z_1, z_2, \ldots, z_{k-1}(z_1 = x + 1, z_2 = z_1 + 1, \ldots, y = z_{k-1} + 1)$
- $y = x - 1$ indica $x = S(y)$, ovvero $x = y + 1$, così come $y = x - k$ indica $x = y + k$
- $last(x)$ indica $\neg\exists y(y > x)$

## Esempi

- Parole non vuote terminanti per $a$: $\exists x(last(x) \wedge a(x))$
- Parole con almeno 3 simboli di cui il terzultimo è $a$
  $\exists x(a(x) \wedge \exists y(y = x + 2 \wedge last(y)))$ Abbreviando: $[\exists x(a(x) \wedge last(x + 2))]$

# Semantica formale

## Semantica dei componenti di una formula

- Dati $w \in \mathbf{I}^+$ e $\mathbf{V}_1$ insieme delle variabili, un assegnamento è una funzione $v_1 : \mathbf{V}_1 \to \{0, 1, \ldots, |w| - 1\}$
  - $w, v_1 \vDash a(x)$ se e solo se $w = uav$ e $|u| = v_1(x)$
  - $w, v_1 \vDash x < y$ se e solo se $v_1(x) < v_1(y)$
  - $w, v_1 \vDash \neg\varphi$ se e solo se $w, v_1 \nvDash \varphi$
  - $w, v_1 \vDash \varphi_1 \wedge \varphi_2$ se e solo se $w, v_1 \vDash \varphi_1$ e $w, v_1 \vDash \varphi_2$
  - $w, v_1 \vDash \forall x(\varphi)$ sse $w, v_1' \vDash \varphi$ per ogni $v_1'$ con $v_1'(y) = v_1(y)$ con $y$ diversa da $x$

## Linguaggio definito da una formula

- $L(\varphi) = \{w \in \mathbf{I}^+ \mid \exists v : w, v \vDash \varphi\}$, con $\varphi$ formula chiusa

# Proprietà della MFO

## Chiusura rispetto ad operazioni

- I linguaggi esprimibili con MFO sono chiusi per unione, intersezione, complemento
  - Basta combinare le formule con $\wedge$, $\vee$, $\neg$
- In MFO non posso esprimere $L = \{a^{2^n}, n \in \mathbb{N}\}$ su $\mathbf{I} = \{a\}$
- MFO è strettamente *meno* potente degli FSA
  - Da una formula in MFO $\varphi$ posso sempre costruire un FSA che riconosce $L(\varphi)$

# Proprietà della MFO

## Chiusura rispetto alla $*$ di Kleene

- I linguaggi definiti da una formula MFO non sono chiusi rispetto alla $*$ di Kleene
  - la formula $a(0) \wedge a(1) \wedge last(1)$ definisce $L = \{aa\}$
  - la $*$-chiusura di $L$ è il linguaggio delle stringhe di $a$ pari
- MFO è in grado di definire i linguaggi *star-free*: sono i linguaggi ottenuti per unione, intersezione, concatenazione e complemento di linguaggi finiti
- Come definire tutti i REG?

# Logica Monadica del Secondo Ordine (MSO)

## Quantificare insiemi di posizioni

- Per avere lo stesso potere espressivo degli FSA basta "solo" permettere di quantificare sui predicati monadici
  - In pratica, quantificare su *insiemi* di posizioni
  - Quantificazione su predicati del prim'ordine $\rightarrow$ logica del secondo ordine
- Ammettiamo formule come $\exists X\ (\varphi)$ con $X$ appartenente all' insieme dei predicati monadici (insiemi di posizioni)
- Convenzione: usamo maiuscole e minuscole
  - Maiuscole per indicare variabili con dominio l'insieme dei predicati monadici
  - Minuscole per indicare variabili $\in \mathbb{N}$

# Semantica

## Assegnamento delle variabili

- L'assegnamento di variabili del 2º ordine (insieme $\mathbf{V}_2$) è una funzione $v_2 : \mathbf{V}_2 \to \wp(\{0, 1, \ldots, |w| - 1\})$
  - $w, v_1, v_2 \vDash X(x)$ se e solo se $v_1(x) \in v_2(X)$
  - $w, v_1, v_2 \vDash \forall X(\varphi)$ se e solo se $w, v_1' \vDash \varphi$ per ogni $v_2'$ con $v_2'(Y) = v_2(Y)$, con $Y$ diversa da $X$

## Esempio

- Possiamo descrivere il linguaggio $L = \{a^{2n}, n \in \mathbb{N} \setminus \{0\}\}$

$$\begin{aligned} \exists P(\forall x( \quad & a(x) \wedge \\ & \neg P(0) \wedge \\ & \forall y(y = x + 1 \Rightarrow (\neg P(x) \Leftrightarrow P(y))) \wedge \\ & (last(x) \Rightarrow P(x)) \qquad\qquad )) \end{aligned}$$

# Da MSO a FSA

## Completare l'equivalenza

- Data una $\varphi$ MSO, si può sempre costruire un FSA che accetta esattamente $L(\varphi)$ (teorema di Büchi-Elgot-Trakhtenbrot)
  - La dimostrazione dell'esistenza è costruttiva: mostra come costruire l'FSA a partire da una formula MSO (non la vediamo per semplicità)
- La classe dei linguaggi definibili via MSO coincide con REG

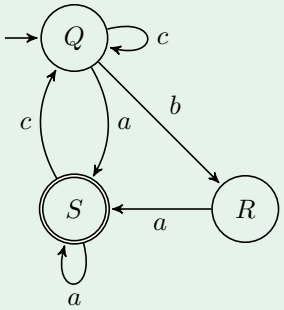
# Da MSO a FSA

## Un approccio costruttivo

- Per ogni $q \in \mathbf{Q}$ dell'FSA, definiamo una variabile $\mathbf{X}_q$ che rappresenta l'insieme di posizioni in una stringa accettata dove l'automa si trova in $q$
- L'automa non è in due stati contemporaneamente: per ogni coppia $\mathbf{X}_i, \mathbf{X}_j$, abbiamo $\neg\exists y(y \in \mathbf{X}_i \wedge y \in \mathbf{X}_j)$
- L'FSA parte da $q_0$ equivale a dire che $\forall x(x = 0 \Rightarrow x \in X_{q_0})$ delle posizioni dei caratteri letti dall' FSA partendo da $q$
- Ogni transizione $\delta(q_i, a) = q_j$ diventa:
  $\forall x, y(y = x + 1 \Rightarrow (x \in \mathbf{X}_i \wedge a(x) \wedge y \in \mathbf{X}_j))$
- L' accettazione via $\delta(q_i, a) \in \mathbf{F}$ diventa: $\forall x(last(x) \Rightarrow \bigvee_{\delta(q_i,a)\in\mathbf{F}}(x \in \mathbf{X}_i \wedge a(x))$

# Da FSA a MSO

## Un esempio pratico




$$
\begin{aligned}
&\exists Q, R, S( \\
&\quad \neg\exists z(\neg(Q(z) \wedge R(z)) \wedge \\
&\qquad \neg(Q(z) \wedge S(z)) \wedge \\
&\qquad \neg(R(z) \wedge S(z))) \quad \wedge \\
&\quad \forall x\,(x = 0 \Rightarrow x \in Q) \quad \wedge \\
&\quad \forall x, y(y = x + 1 \Rightarrow \\
&\qquad (Q(x) \wedge c(x) \wedge Q(y)) \vee \\
&\qquad (Q(x) \wedge b(x) \wedge R(y)) \vee \\
&\qquad (Q(x) \wedge a(x) \wedge S(y)) \vee \\
&\qquad (R(x) \wedge a(x) \wedge S(y)) \vee \\
&\qquad (S(x) \wedge a(x) \wedge S(y)) \vee \\
&\qquad (S(x) \wedge c(x) \wedge Q(y))) \quad \wedge \\
&\quad \forall x\,(last(x) \Rightarrow \; Q(x) \wedge a(x) \vee R(x) \wedge a(x) \vee \\
&\qquad\qquad S(x) \wedge a(x)))
\end{aligned}
$$

# Logica per definire proprietà dei programmi

## Un formalismo per definire gli effetti

- Specifica di un algoritmo di ricerca: la variabile found $\in \{0, 1\}$ deve valere 1 se e solo se esiste un elemento dell' array $a$ di $n$ elementi uguale all' elemento $x$ cercato
  - found $\Leftrightarrow \exists i(a[i] = x \wedge 0 \leq i \leq n - 1)$
- Specifica di un algoritmo di inversione out-of-place di un array $a$ in un array $b$
  - $\forall i, (0 \leq i \leq n - 1 \Rightarrow b[i] = a[n - 1 - i])$

# Più in generale

## Pre- e Post-condizioni

- Precondizioni: insieme di condizioni che devono essere vere prima dell'esecuzione di un programma $P$ affinchè sia vero un insieme di fatti (*post-condizioni*) dopo l' esecuzione
- Esempio: ricerca di un elemento $x$ in un array ordinato $a$

  Pre $\{\forall i, (0 \leq i \leq n-2 \Rightarrow a[i] \leq a[i+1])\}$

  - Esecuzione del programma `P`

  Post $\{\text{found} \Leftrightarrow \exists i(a[i] = x \wedge 0 \leq i \leq n-1)\}$
- Controesempio: un algoritmo di ricerca binaria non garantirebbe post se pre fosse semplicemente $\{True\}$
- N.B. le pre e post precedenti non implicano che `P` sia un algoritmo di ricerca binaria: una ricerca lineare funziona ugualmente

# Un ulteriore esempio

## Ordinamento di array di $n$ elementi senza ripetizioni

Pre $\{\neg\exists i, j\ (\ 0{\leq}i{\leq}n{-}1\ \wedge\ 0{\leq}j{\leq}n{-}1\ \wedge\ a[i]{=}a[j]\ \wedge\ i{\neq}j)\}$

- Esecuzione di `ORD`

Post $\{\forall i, (\ 0{\leq}i{\leq}n{-}2 \Rightarrow a[i] \leq a[i+1])\}$

## "Buone" specifiche

- É una specifica "adeguata"?
- La specifica agisce come un "contratto" con chi deve sviluppare `ORD`, così come con chi usa il programma sviluppato

# Ordinamento di array di $n$ elementi senza ripetizioni

## Una specifica più accurata

Pre $\{ \neg\exists i,j\ (\ 0{\leq}i{\leq}n{-}1\ \wedge\ 0{\leq}j{\leq}n{-}1\ \wedge\ a[i]{=}a[j]\ \wedge\ i{\neq}j)\ \wedge\ \forall i(\ 0{\leq}i{\leq}n{-}1 \Rightarrow a[i] = b[i])\ \}$

- Esecuzione di `ORD`

Post $\{\ \forall i,(\ 0{\leq}i{\leq}n{-}2 \Rightarrow a[i]{\leq}a[i+1]) \wedge$
$\forall i(\ 0{\leq}i{\leq}n{-}1 \Rightarrow \exists j(\ 0{\leq}j{\leq}n{-}1\ \wedge a[i] = b[j])) \wedge$
$\forall j(\ 0{\leq}j{\leq}n{-}1 \Rightarrow \exists i(\ 0{\leq}j{\leq}n{-}1\ \wedge a[i] = b[j]))\ \}$

## "Buone" specifiche

- Se eliminiamo la prima porzione della Pre, la specifica è ancora valida?
- La specifica data è un "buon" modello anche per l' ordinamento di una lista o un file?

# Verso metodi e linguaggi di specifica

## Verifiche di correttezza di implementazioni

1. Specificare i requisiti di un algoritmo in un'opportuna logica
2. Implementare l'algoritmo in un opportuno linguaggio
3. Ottenere la correttezza dell'implementazione come dimostrazione (automatizzata) di un teorema

## Logica come descrizione di "dati"

- É possibile scegliere un'opportuna logica per descrivere un insieme di concetti
  - e.g., RDF per pagine web, logiche descrittive per dati biomedici
- Se la logica è opportuna (= è possibile calcolare la verità di un dato teorema) possiamo automatizzare la validazione di nostre deduzioni su vaste quantità di dati